Technical Assessment

Project Expense & Claim Tracker

Full-Stack Engineering — Lovable + Python Backend

Field	Details
Role	Full-Stack Developer
Time Allowance	2 days
Required Stack	Lovable (frontend) Or preferred AI software + Python backend API (FastAPI or Flask)
Domain	Project cost management — track expenses (claimed / not-claimed) and income
Base Currency	MYR (Ringgit Malaysia).
Key Feature	Extract expenses from a Malaysian bank credit card statement (PDF)
Frontend Workflow	Build UI in Lovable (https://lovable.dev)→ export code → wire to Python backend
Submission	GitHub repository OR hosted on Google Cloud (GCP) for review

1. Overview

Thank you for taking the time to work through this assessment. The goal is to build a small but realistic project cost management system that reflects the kind of work you would do on our team.

We care less about a fully polished product and more about how you think, structure code, make tradeoffs, and communicate. Submit what you have at the end of the window even if incomplete — partial, thoughtful work beats rushed, broken work.

1.1 Business Context

Our team runs multiple projects. Each project incurs expenses (travel, equipment, meals, software, etc.) and receives income. Some of these expenses are later claimed back (by the business, a client, a

reimburser — it does not matter who for the purpose of this assessment), and some are not. We need to know, at any point, which expenses have been claimed and which are still outstanding.

Today the team tracks all of this in messy spreadsheets and manually copies rows from credit card PDF statements — which is slow and error-prone. You are building the first version of a system that replaces that workflow.

Scope clarification — expenses vs. income: the PDF credit card statement is used **only** as a source for expenses, and each imported expense must be assigned to a specific project and flagged as claimed or not-claimed. Income is handled separately and is **always entered manually by the user** under a specific project — do not attempt to derive income from the PDF.

2. What to Build

A web application with the following core capabilities. You do not need to build everything — read the Scope & Tiers section before you start.

2.1 Core Entities

- **Project** — name, description, start/end dates, total budget (RM), status (active / on hold / closed).
- **Expense** — project, date, vendor/description, amount, currency, category, is_claimed (boolean), claimed_date (optional), notes, source reference (e.g. which PDF statement it came from, if any). Expenses may enter the system either through manual entry or through PDF statement import.
- **Income** — project, date, source, amount, currency, notes. Manually input.

Note on claims: keep it simple. An expense is either claimed or not. The user should be able to flip that flag freely from the UI, and (if claimed) optionally record when. You do not need to model 'who it was claimed from' or multi-step approval workflows for this assessment.

2.2 Functional Requirements

Frontend (built in Lovable)

Workflow: design and build the UI inside Lovable, then export/download the generated code (Lovable supports exporting to a GitHub repo or downloading the source) and wire it up to your Python backend API. We want to see both the Lovable-built UI and the integrated codebase in your submission.

- Dashboard showing per-project totals: total income, total expenses, net position, total claimed, total not-claimed (i.e. outstanding).
- Project list and project detail views.
- Expense list with filters (project, date range, category, claim status) and the ability to toggle the claimed / not-claimed flag directly from the list or detail view.
- Income list with create / edit / delete, scoped to a specific project. This is the **only** way income enters the system — all income is manually entered.

- **PDF upload page** — user uploads a credit card statement PDF, the system extracts transactions, and the user reviews / edits / confirms before the expenses are saved to a project.

Backend (Python API)

- REST API using FastAPI or Flask. FastAPI is preferred but not required.
- Persistent storage — SQLite or Postgres
- CRUD endpoints for Project, Expense, Income.
- Endpoint to toggle an expense's claimed / not-claimed status (single expense and bulk).
- Aggregation endpoint(s) that power the dashboard in a single request per project (do not force the frontend to compute totals by fetching every row).
- **PDF ingestion endpoint** — accepts a PDF credit card statement, parses it, returns a structured list of candidate transactions (date, description, amount). Persisting is a separate confirm step.
- Input validation and sensible HTTP error responses (not bare 500s).
- Basic automated tests — at minimum cover the PDF parser and the dashboard aggregation logic.

PDF Ingestion — Details

This is the feature we are most interested in seeing you design. A sample Malaysian bank credit card statement PDF is provided with this brief (sample_credit_card_statement.pdf). Your parser should handle this sample correctly. Every transaction imported from the PDF is an expense tied to a project the user selects at upload time — income is never derived from the PDF and is always entered manually elsewhere in the app.

- Identify each transaction row — posting date, transaction date, merchant/description, amount in RM.
- **Handle the local 'CR' suffix convention** — amounts ending in ' CR' are credits/payments (e.g. 'RM 1,856.20 CR'). These are NOT expenses and must not be imported as such.
- Ignore the Account Summary block at the top (Previous Balance, New Charges, Finance Charges, Minimum Payment Due, etc.) — those are statement totals, not transactions.
- Ignore sub-total rows at the bottom of the transaction list ('Sub-Total Charges', 'Sub-Total Payments & Credits').
- Handle overseas transactions correctly — the primary amount is already converted to RM; the Overseas Transaction Details section provides the original foreign amount and FX rate. Decide (and document) whether you store the RM amount, both, or something else.

3. Function Scope

- Projects, Expenses, Income CRUD end-to-end (Lovable-built UI + exported code + Python API + DB).
- Per-project dashboard showing total income, total expenses, net, total claimed, total notclaimed.
- Toggle an expense between claimed and not-claimed from the UI.
- PDF upload that at minimum extracts transaction text from the sample PDF — even a rough list is fine. Preview + manual confirm before save.

- Dashboard breakdown by category and by claim status, not just totals.
- Reasonable API design — consistent resource naming, status codes, error shape.
- Duplicate detection when re-uploading a statement that overlaps a previous one.

4. What We Provide

- **sample_credit_card_statement.pdf** — a representative Malaysian bank statement (fictional 'Mayhill Bank Berhad') with 30 transaction rows across two pages, including charges, a payment marked 'CR', a zero-amount waived fee, a finance charge, an overseas USD transaction, and nontransaction summary sections. Your parser must handle this file.
- This brief.

You may use any public libraries, frameworks, AI coding tools, or LLM APIs you want. If you used an AI assistant, briefly note in the README where it helped and where it did not — we are not scoring this against you, we want to understand your process.

5. Deliverables

Submit your work via ONE of the following two routes.

Option A — GitHub Repository

1. **A single GitHub repository** (public, or private with access shared to the reviewer) containing:

- /backend — your Python API (with requirements.txt or pyproject.toml, schema/migrations, tests, README for local setup).
- /frontend — the code exported / downloaded from Lovable, with any modifications you made to integrate with the backend.
- A top-level README.md covering: tier targeted, architecture overview, how to run both parts locally, environment variables needed, trade-offs, and what you would do next with one more week.

Option B — Hosted on Google Cloud (GCP)

Preferrable, host the running system for review, deploy to GCP and share:

- A working URL for the frontend (Cloud Run, App Engine, Firebase Hosting, or GCS + Cloud CDN — your choice).
- A working URL for the backend API (Cloud Run is the most straightforward for a Python API; App Engine / GKE are fine if you prefer).
- The source code — either in a GitHub repo linked from your submission, or zipped and attached. We still need to read it.
- A README covering the same points as Option A, plus brief notes on how the GCP deployment is set up (services used, how to redeploy).

Note: if you go with Option A and cannot deploy anywhere, the README must have dead-simple local setup instructions. We should be able to go from 'git clone' to a running app in under 10 minutes.

6. FAQ

Do I have to use Lovable for the frontend? Yes — for the initial build. The workflow we want to see is: design and build in Lovable, then export the generated code (Lovable has a GitHub sync and a download option), then modify it as needed to integrate with your Python backend. Submit both so we can see the Lovable-built UI and the final integrated codebase.

Can I heavily modify the Lovable-exported code? Yes. Once it is out of Lovable, it is just code — clean it up, refactor it, restructure it as you see fit. Be ready to explain your changes in the video walkthrough.

GitHub or GCP — which should I pick? Pick GitHub if you want to minimise ops work. But we will prefer GCP as we will require some knowledge to work on GCP as well. if you are comfortable deploying and want the live URL to speak for itself.

Postgres or SQLite? Either. SQLite is fine for a take-home. Just don't forget to include the schema/migrations. If you deploy to GCP, Cloud SQL (Postgres) is an easy choice.

Good luck — we're looking forward to seeing what you build.